


**UNIVERSIDAD DEL NORTE**

Maestría en Estadística Aplicada

Machine Learning

**Daniel Carbonó Domínguez**

**Luis Pérez**

Octubre de 2026

CLASIFICACIÓN BINARIA SUPERVISADA

# Predicción del incumplimiento de órdenes de compra en la agroindustria de la palma de aceite

406.126 posiciones de 104.614 órdenes de compra, 2021-2026

Repositorio github.com/ddcarbono/otif-compras-ml

Libro ddcarbono.github.io/otif-compras-ml

Tablero otif-compras-ml.onrender.com

# Uno de cada tres pedidos llega tarde, y se sabe cuando ya pasó

CRITERIO 1 · EL PROBLEMA

**33,0 %**
de las posiciones llega tarde

**41 %**
en industrial; 25 % en ganaderia

**71 %**
de las tardías de 2026, marcando el 38 %

## Por qué importa

El comprador hoy se entera del retraso cuando la mercancía no llega. Con miles de posiciones abiertas, no puede seguirlas todas: necesita saber, el día en que libera la orden, cuáles vigilar.

## Qué dice la literatura

En fabricantes de maquinaria y equipos llega tarde del 13 % al 14 % de los pedidos (Brintrup y otros, 2020; Steinberg y otros, 2023): aquí es más del doble.

Una revisión de 26 estudios encuentra que 16 parten las filas al azar; reevaluado sin fuga y en el tiempo, un AUC de 0,96 baja a 0,73 (Raval, 2026). Este trabajo evalúa siempre hacia adelante.

## Objetivos

General: ordenar las posiciones por riesgo de retraso el día de la liberación y señalar cuáles seguir.
Específico 1: comparar 7 modelos × 4 balanceos × 4 optimizadores con el mismo presupuesto.
Específico 2: medir cuánto cuesta decidir con entidades nuevas (proveedor o material sin historia).
Específico 3: dejar el modelo servido en un tablero con predicción en vivo.

Fuente: libro, introducción y literatura; capítulo 4 (modelo base): data/modelos/metricas.csv

# Estimar la probabilidad de llegar tarde y decidir con un umbral

CRITERIO 1 · FORMULACIÓN MATEMÁTICA

### Unidad: la posición i, el día t en que se libera su orden

$$x_i \in \mathbb{R}^{136}, \quad y_i \in \{0, 1\}$$

x: 136 predictoras conocidas en t. y = 1 si la primera entrada de mercancía llega más de 1 día hábil después de la fecha prometida vigente al liberar.

### El modelo estadístico: y | x es Bernoulli con probabilidad p(x)

$$p(x) = P(y = 1 \mid x), \qquad \hat{y} = \mathbb{1}\{\hat{p}(x) \geq \tau\}$$

Se estima p(x) minimizando la log-pérdida, que es menos la log-verosimilitud de Bernoulli; la decisión ŷ compara p̂ con un umbral τ.

$$\ell(y, F) = -\log\left[\hat{p}^{\,y}(1 - \hat{p})^{1-y}\right], \qquad \hat{p} = \sigma(F)$$

**El umbral τ se elige en la validación 2025, maximizando F1; la prueba 2026 se usa una sola vez y no elige nada.**

Métricas: AUC para ordenar por riesgo (no depende de τ) y F1 para la decisión.

### Tarea

Clasificación binaria supervisada: la etiqueta se observa después, cuando entra la mercancía, y el modelo aprende de las órdenes ya cerradas.

### Por qué no regresión

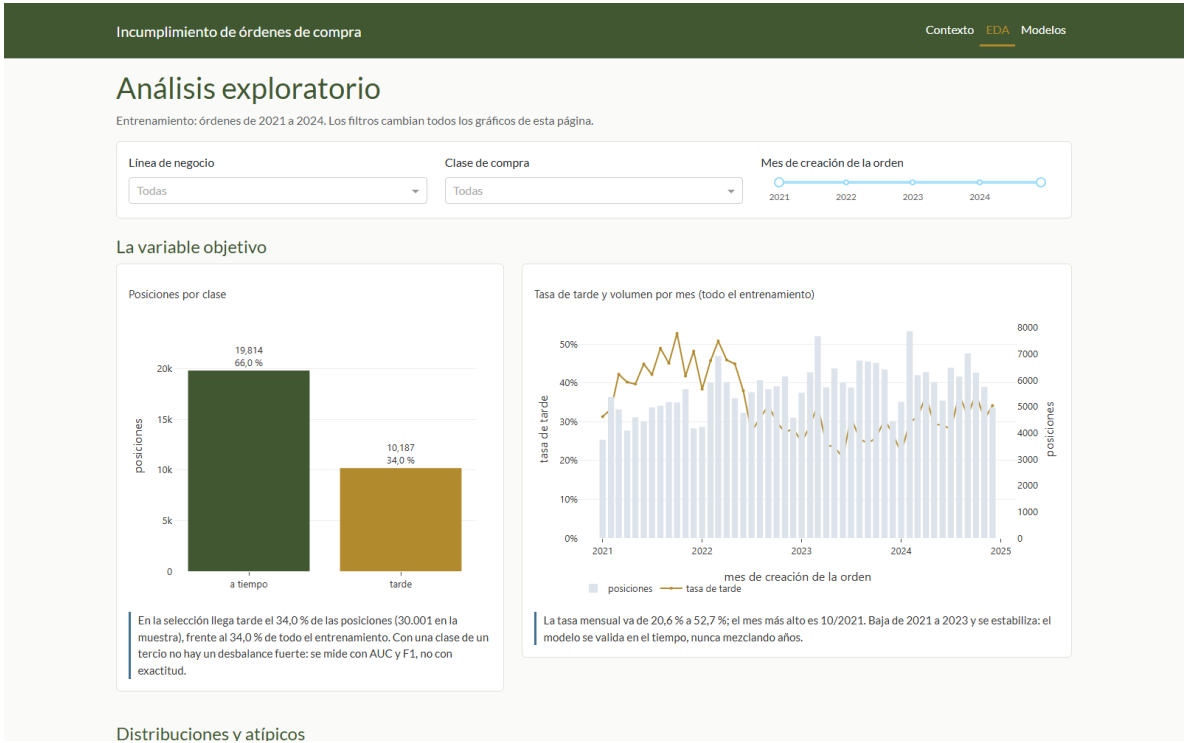
La decisión del comprador es binaria: seguir la orden o no. Los días de retraso tienen colas muy largas y se usan como contexto; el modelo original del Entregable 3 los aprovecha con cortes ordenados.

Fuente: libro, capítulos 1 y 2a (definición de la etiqueta y de la población)

# Tablero: el contexto y la exploración, con filtros

## CRITERIO 2 · PANTALLA EN VIVO

captura de respaldo, por si el tablero no carga




**EN VIVO**

otif-compras-ml.onrender.com

1. Contexto: la fuente (SAP, 2021-2026), la población y la etiqueta.
2. EDA: el objetivo por clase y la tasa por mes.
3. Filtrar una línea de negocio: todo cambia con ella (de 25 % a 41 % de tarde).
4. Bajar a distribuciones y atípicos.

Fuente: tablero, páginas Contexto y EDA (otif-compras-ml.onrender.com)

# De SAP a 406.126 posiciones con una etiqueta y 136 predictoras

## CRITERIO 2 · FUENTE, ESTRUCTURA Y PARTICIÓN

**SAP**
compras 2021-2026

**490.310**
posiciones con plazo

**406.126**
población

**etiqueta**
> 1 día hábil tarde

**136**
predictoras al liberar

| partición | uso | posiciones | órdenes | tarde |
|---|---|---|---|---|
| **2021-2024** | entrenar | 271.391 | 70.233 | 34,0 % |
| **2025** | elegir umbral | 82.794 | 21.797 | 30,8 % |
| **2026** | probar una vez | 51.941 | 12.584 | 31,5 % |

Las órdenes son la unidad independiente: 3,9 posiciones por orden. Ninguna orden cruza particiones y toda validación cruzada agrupa por orden.

La partición es por la fecha de la orden, no al azar (diapositiva 8).

### Las 136 predictoras

83 numéricas: plazo, valor, liberación, historial del proveedor, stock, tasa de cambio.
42 categóricas de pocos valores: línea, clase, moneda, tipo de material.
11 entidades: proveedor, material, comprador, liberador, centro de costo y otras.
Solo lo que se conoce el día de la liberación.

Fuente: libro, capítulo 2a (datos); diccionario de variables en data/diccionario.csv

# Colas largas, vacíos con causa y atípicos que llegan más tarde

## CRITERIO 2 · DESCRIPTIVA, FALTANTES Y ATÍPICOS

| variable | mediana | p99 | asimetría |
|---|---|---|---|
| **Valor para la liberación** | 24.807,3 | 6.967.970 | 294,2 |
| **Precio de la solicitud** | 499,0 | 364.340 | 247,5 |
| **Razón cantidad orden/solicitud** | 1,0 | 1 | 238,5 |
| **Razón al último precio del material** | 1,0 | 6 | 213,5 |
| **Razón precio orden/solicitud** | 1,0 | 78.327 | 190,0 |
| **Plazo original al crear** | 20,0 | 177 | 148,7 |

Por eso signo(x)·log(1 + |x|) antes de los modelos lineales: comprime las colas sin cambiar el orden. A los árboles no les cambia nada, porque solo usan el orden.

### Atípicos multivariados (Isolation Forest)

40,4 % de tarde en las atípicas contra 33,7 % en el resto: son señal, no ruido, y no se borran.

| variable | vacíos | Δ tasa (pp) |
|---|---|---|
| **mc_aprov_especial** | 93,6 % | 0,3 |
| **imp_categoria_orden** | 88,8 % | -0,3 |
| **razon_cantidad_stock** | 76,3 % | 2,3 |
| **mat_razon_ultimo_precio** | 51,9 % | -0,8 |
| **imp_n** | 50,3 % | 2,9 |
| **imp_a_activo** | 50,3 % | 2,9 |

### Faltantes estructurales

7 variables con más de 50 % de vacíos, y no son azar: faltan porque la orden no tiene solicitud o no tiene imputación. Se marcan con indicadores tiene_* y se imputa la mediana; Δ tasa es la diferencia de retraso entre filas con y sin dato.

Fuente: libro, capítulo 2b (EDA): resultados/eda/univariado_numericas.csv, faltantes.csv, atipicos_if.csv

# Ninguna variable sola pasa de 0,75 de AUC, y ninguna es fuga

## CRITERIO 2 · RELACIONES Y FUGA




El plazo hábil al liberar separa más (0,732): es la causa más directa del retraso y se conoce al liberar.

### Auditoría de fuga

Cada grupo de variables, revisado contra el día de la liberación. Retiradas: los días de retraso, las fechas de entrada y de factura, los cambios de fecha posteriores y los maestros que el planificador ajusta después de ver retrasos.

### Entidades: tasa encogida hacia su nivel superior

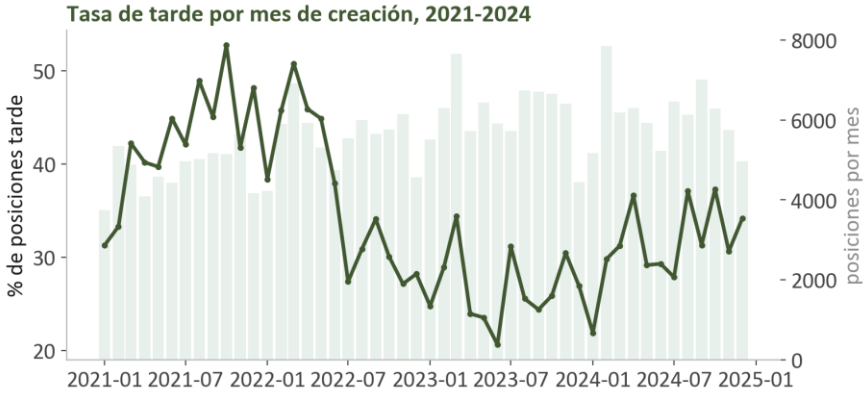
$$\hat{p}(v) = \frac{\text{retrasos}(v) + M\hat{p}(\text{padre}(v))}{n(v) + M}, \qquad M = 20$$

Se calcula fuera de pliegue y agrupada por orden: una posición no ve su propio objetivo ni el de su orden. Un valor nuevo recibe la tasa de su padre (el proveedor, la de su región).

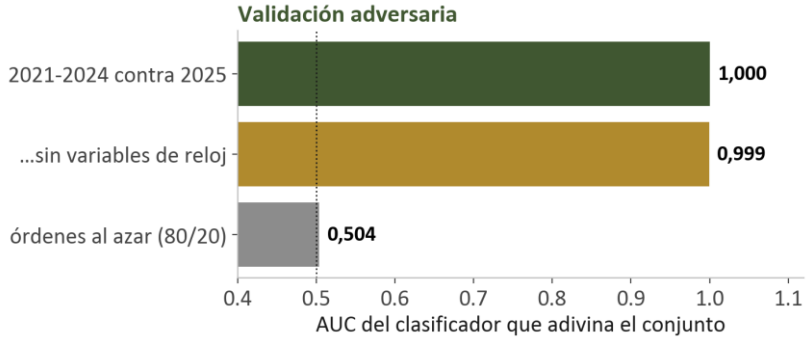
Fuente: libro, capítulos 2b y 2c: resultados/eda/asociacion.csv; src/datos.py

# El tiempo importa: partir al azar mediría con datos ya vistos

## CRITERIO 2 · DERIVA Y PARTICIÓN




No es estacionaria: ADF no rechaza la raíz unitaria (p = 0,70) y KPSS rechaza la estacionariedad (p = 0,026). Proveedores nuevos: 6,6 % en 2025 y 11,1 % en 2026.




**Validación adversaria: un XGBoost intenta adivinar de qué conjunto viene cada posición.**

Al azar no puede (0,504). Por fecha acierta siempre (1,000), y sigue en 0,999 sin las variables que fechan la fila: muchas derivas pequeñas (IVA, región del centro, mezcla de líneas) bastan para reconocer el año.

**Por eso: partición cronológica, validación por ventanas crecientes y reentrenar con 2021-2025 antes de la prueba.**

Fuente: libro, capítulo 2c (tiempo y fuga): resultados/eda/ y resultados/similitud/auc.csv

# XGBoost: el log-odds como suma de árboles

CRITERIO 3 · FORMULACIÓN DEL ALGORITMO

## El modelo

$$F(x) = \sum_{m=1}^{M} f_m(x), \qquad \hat{p}(x) = \sigma(F(x)) = \frac{1}{1+e^{-F(x)}}$$

Cada árbol f_m reparte el espacio de las predictoras en T hojas y da a cada hoja un peso w_j. Los árboles se construyen en secuencia: el m corrige lo que los m − 1 anteriores todavía predicen mal.

## La pérdida: menos la log-verosimilitud de Bernoulli

$$\ell(y, F) = -y \log \sigma(F) - (1-y)\log(1-\sigma(F))$$

Es convexa en F y penaliza sin límite una probabilidad segura y equivocada: por eso produce probabilidades, no solo un orden.

## El principio: descenso por gradiente en el espacio de funciones

En vez de mover parámetros, cada iteración agrega una función (un árbol) en la dirección que más baja la pérdida (Friedman, 2001). XGBoost usa además la curvatura: un paso de Newton (Chen y Guestrin, 2016).

### Boosting

Árboles en secuencia, cada uno sobre los errores del anterior: baja el sesgo. Pide regularización para no sobreajustar.

### Bosque aleatorio

Árboles independientes y promediados: baja la varianza. Fue el segundo de los 112; la diferencia está en corregir el error en vez de promediarlo.

Fuente: libro, capítulo 7 (fundamentos); Friedman (2001), Chen y Guestrin (2016)

# Cada árbol minimiza una aproximación de segundo orden

CRITERIO 3 · PRINCIPIO DE OPTIMIZACIÓN

**El objetivo del árbol m: la pérdida más una penalización de su complejidad**

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} \ell(y_i,\, F_{m-1}(x_i) + f(x_i)) + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

**Taylor de segundo orden alrededor de la predicción actual F_{m−1}**

$$\mathcal{L}^{(m)} \approx \sum_{i} \left[ g_i f(x_i) + \tfrac{1}{2} h_i f(x_i)^2 \right] + \gamma T + \tfrac{1}{2}\lambda \sum_{j} w_j^2$$

**Gradiente y hessiano de la log-pérdida: la derivación**

$$\sigma'(F) = \sigma(1-\sigma) \;\Rightarrow\; \frac{\partial \ell}{\partial F} = -y(1-\sigma) + (1-y)\,\sigma = \sigma - y, \qquad \frac{\partial^2 \ell}{\partial F^2} = \sigma(1-\sigma)$$

$$g_i = \hat{p}_i - y_i, \qquad h_i = \hat{p}_i\,(1-\hat{p}_i)$$

### g_i, el gradiente

El residuo con signo: positivo si el modelo sobreestima el riesgo de una posición que llegó a tiempo.

### h_i, el hessiano

La varianza de Bernoulli: grande donde el modelo duda (p̂ cerca de 0,5) y pequeño donde ya está seguro.

Fuente: libro, capítulo 7; Chen y Guestrin (2016)

# Peso de hoja y ganancia de corte en forma cerrada

CRITERIO 3 · CÓMO SE CONSTRUYE CADA ÁRBOL

**Peso óptimo de la hoja j**

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Con G_j y H_j la suma de g y h en la hoja, la aproximación es una parábola en w_j: su mínimo es un paso de Newton, el gradiente sobre la curvatura, encogido por λ.

**Búsqueda por histograma (hist)**

Cada predictora se discretiza en a lo sumo 256 tramos y la ganancia se evalúa solo en sus bordes: buscar el mejor corte es recorrer un histograma, no ordenar las filas.

**Ganancia de partir una hoja en izquierda (L) y derecha (R): se corta solo si es positiva**

$$\text{ganancia} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

**Paso corto: cada árbol entra encogido por la tasa de aprendizaje**

$$F_m = F_{m-1} + \eta f_m$$

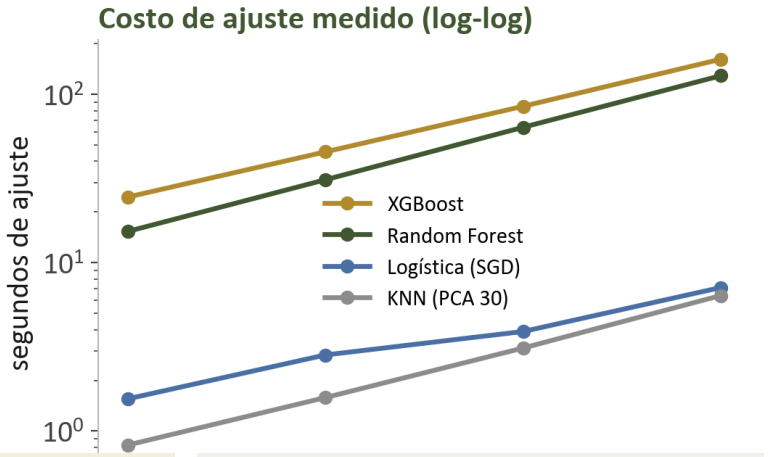
**Contraste: la logística** minimiza la misma log-pérdida con una sola recta, convexa y con un único óptimo. No ve interacciones ni umbrales sin construirlos; XGBoost los encuentra con los cortes.

Fuente: libro, capítulo 7; Chen y Guestrin (2016)

# El ajuste crece casi lineal en n (exponente 0,91)

## CRITERIO 3 · SUPUESTOS, REGULARIZACIÓN, COMPLEJIDAD Y GENERALIZACIÓN

| hiperparámetro | en la fórmula | qué controla |
|---|---|---|
| **n_estimators** | M | capacidad y costo |
| **learning_rate** | η | pasos cortos, mejor generalización |
| **max_depth** | T ≤ 2^D | interacciones de hasta D variables |
| **min_child_weight** | H_j ≥ c | hojas con suficiente evidencia |
| **subsample, colsample** | filas y columnas | azar que decorrelaciona árboles |
| **reg_lambda, gamma** | λ, γ | encoge pesos, cobra hojas |




### Supuestos

Filas independientes dado x (por eso los pliegues agrupan por orden); la relación de 2021-2025 sigue valiendo en 2026; no supone linealidad ni normalidad.

### Complejidad

Ajuste O(M·D·n·d') por histograma; predicción O(M·D). Medido: 162 s con 311.865 filas; 16,8 ms por cada 1.000 predicciones.

### Generalización

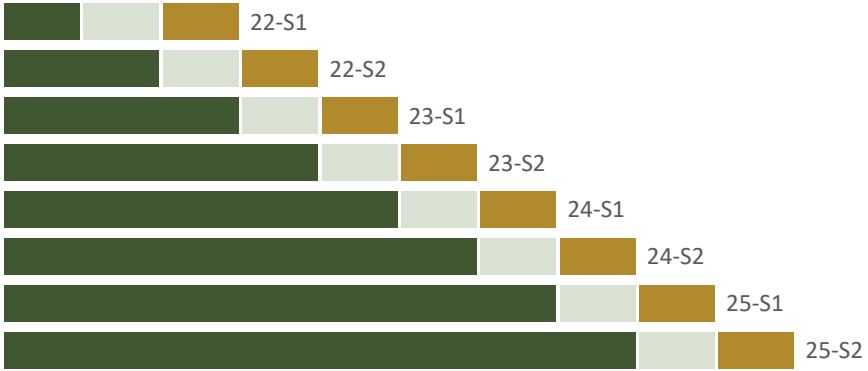
Regularización más validación por ventanas en el tiempo; una entidad nueva recibe la tasa de su nivel superior.

Fuente: libro, capítulo 7: resultados/complejidad.csv (tiempos medidos en srvgdai)

# Hacia adelante en el tiempo, con el mismo presupuesto

**CRITERIO 4 · DISEÑO EXPERIMENTAL**

$$7 \times 4 \times 4 = 112 \text{ combinaciones} \times 8 \text{ bloques} = 896 \text{ trabajos}$$




**verde: entrena** gris: valida adentro (elige hiperparámetros) **oro: evalúa (bucle externo)**

Bucle externo: 8 semestres, cada uno predicho con todo lo anterior. Bucle interno: el último semestre de cada ventana elige los hiperparámetros. La prueba 2026 no entra en nada de esto.

### 4 balanceos

Ninguno, SMOTE, ADASYN y pesos de clase, siempre dentro del pipeline y solo sobre el entrenamiento de cada ventana: lo sintético nunca llega a la evaluación.

### 4 optimizadores, mismo presupuesto

20 evaluaciones por combinación y bloque para Grid, Random, Optuna (TPE) y DEAP (genético). Medias y desviaciones salen de los 8 bloques externos.

Fuente: libro, capítulo 5 (las 112): src/validacion.py, src/experimentos.py

# Implementación: un pipeline, un espacio de búsqueda y pruebas

## CRITERIO 4 · IMPLEMENTACIÓN E HIPERPARÁMETROS

| hiperparámetro | espacio | elegido (8 bloques) |
|---|---|---|
| **n_estimators** | 100 a 600 | 266 a 587 |
| **learning_rate** | 0,01 a 0,3 (log) | 0,02 a 0,12 |
| **max_depth** | 3 a 10 | 4 a 10 |
| **subsample** | 0,5 a 1 | 0,52 a 0,94 |
| **colsample_bytree** | 0,3 a 1 | 0,31 a 0,86 |
| **min_child_weight** | 1 a 100 (log) | 8,91 a 38,19 |

Con un semestre de entrenamiento gana un modelo de muchos árboles poco profundos; con más datos la profundidad sube a 7-10 y el submuestreo baja. Cerca del óptimo la superficie es plana: configuraciones distintas dan AUC a milésimas.

**Reproducible:** semilla 2026; un JSON por trabajo con parámetros y métricas; 201 horas de CPU en el servidor del grupo, 30 trabajos a la vez.

### Un solo pipeline de scikit-learn

preprocesador (log, imputación, one-hot, codificación por objetivo) → balanceo → modelo;
el preprocesador se ajusta solo con el entrenamiento de cada ventana.

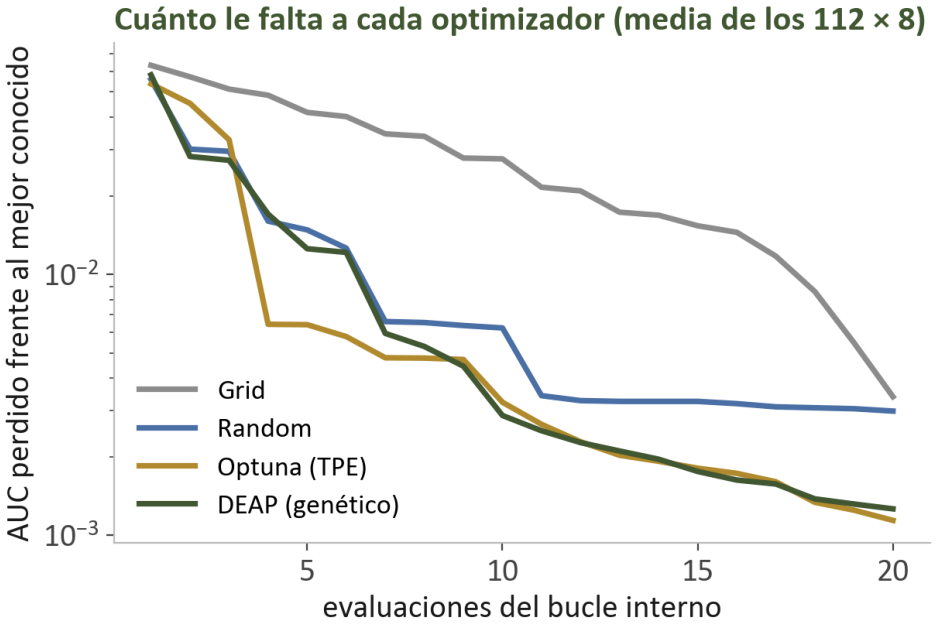
### Pruebas e integración continua

57 pruebas con pytest (codificación fuera de pliegue, series, modelo servido, tablero, que no se publiquen identificadores) que corren en GitHub en cada cambio.

Fuente: src/experimentos.py (ESPACIOS), resultados/experimentos/trabajos/*.json, tests/

# Optuna llega antes; con 20 evaluaciones empatan los cuatro

CRITERIO 4 · PANTALLA EN VIVO: LAS 112




| optimizador | AUC perdido a 5 evaluaciones | a 20 |
|---|---|---|
| **Optuna** | 0,0064 | 0,0011 |
| **DEAP** | 0,0125 | 0,0013 |
| **Random** | 0,0148 | 0,0030 |
| **Grid** | 0,0417 | 0,0034 |

**EN VIVO**

otif-compras-ml.onrender.com

1. Modelos → Las 112 combinaciones.
2. Tabla de líderes: las mejores combinaciones, con media ± desviación del bucle externo.
3. Mapa balanceo × optimizador de XGBoost: el balanceo casi no mueve el AUC.

Fuente: tablero, página Modelos (otif-compras-ml.onrender.com); data/modelos/curvas_optimizador.csv

# XGBoost gana los 8 bloques: AUC 0,852 contra 0,843

## CRITERIO 5 · MÉTRICAS, LÍNEA BASE Y COMPARACIÓN

| modelo (mejor combinación) | AUC ± desv. | PR-AUC | F1 | ms / 1.000 |
|---|---|---|---|---|
| **XGBoost · sin balanceo · Optuna** | 0,852 ± 0,020 | 0,778 | 0,676 | 17,3 |
| **Random Forest · SMOTE · Grid** | 0,843 ± 0,023 | 0,761 | 0,660 | 31,8 |
| **Logística (SGD) · pesos · Grid** | 0,839 ± 0,025 | 0,751 | 0,663 | 0,7 |
| **SVM lineal (SGD) · pesos · Random** | 0,839 ± 0,024 | 0,749 | 0,659 | 0,7 |
| **Árbol · sin balanceo · Optuna** | 0,810 ± 0,020 | 0,715 | 0,634 | 0,8 |
| **KNN (PCA 30) · SMOTE · Optuna** | 0,781 ± 0,044 | 0,665 | 0,601 | 1.414,6 |
| **Naive Bayes · SMOTE · DEAP** | 0,735 ± 0,069 | 0,566 | 0,556 | 29,4 |

AUC ordena por riesgo sin depender del umbral; PR-AUC atiende a la clase de interés; F1 mide la decisión al umbral; Brier y ECE dicen si la probabilidad es creíble.

| prueba 2026 (línea base) | AUC | F1 | Brier |
|---|---|---|---|
| **regla del proveedor** | 0,670 | 0,509 | 0,202 |
| **regresión logística** | 0,825 | 0,649 | 0,149 |

### Cómo leer la tabla

Media y desviación de los 8 bloques externos: cada cifra se midió en un semestre que el modelo no vio. La línea base son la regla del proveedor (su tasa previa de retraso) y la logística, medidas en 2026. KNN predice 80 veces más lento que XGBoost: compara contra todo el entrenamiento.

### Pendiente: evaluación final 2026

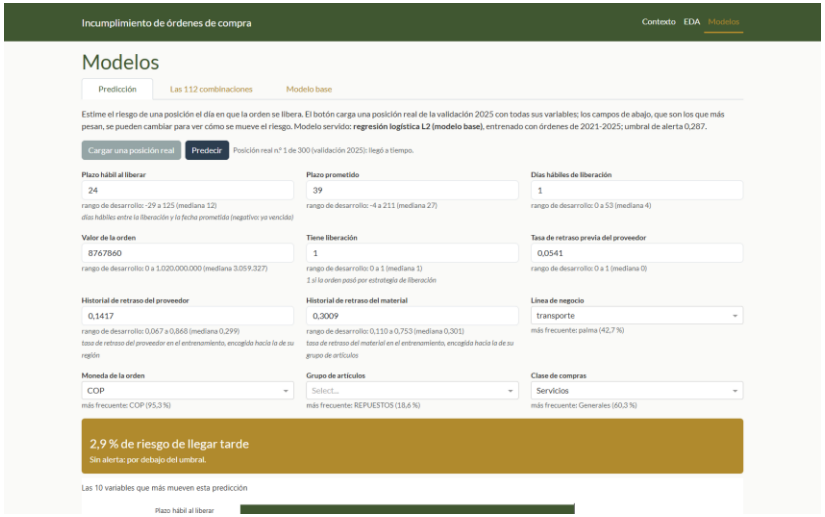
XGBoost en la prueba 2026 y la tabla de brecha validación-prueba de cada modelo.

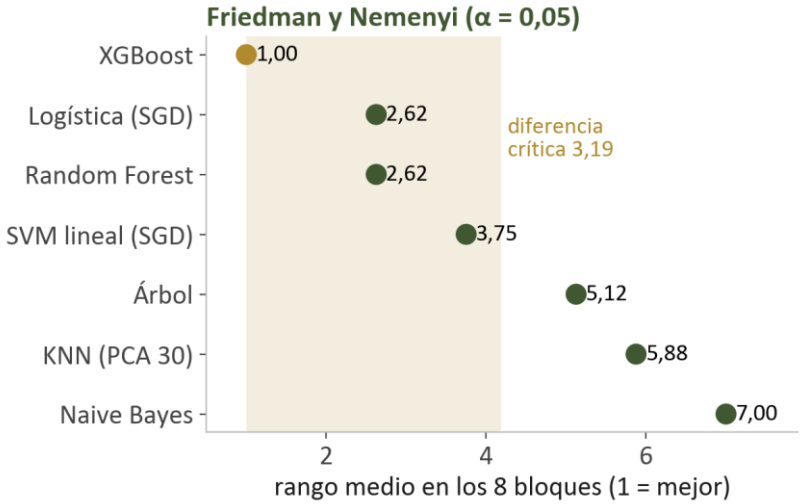
Fuente: libro, capítulos 4 y 5: data/modelos/lideres.csv, metricas.csv

# Tablero: una predicción en vivo, y la prueba de diferencias

## CRITERIO 5 · PANTALLA EN VIVO

captura de respaldo, por si el tablero no carga




**En vivo:** cargar una posición real, cambiar el plazo hábil al liberar y ver cómo se mueven el riesgo y sus contribuciones.

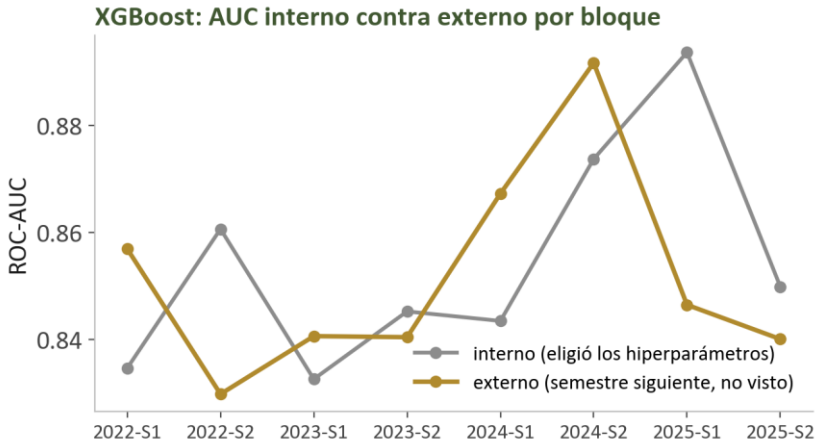
**Friedman: $\chi^2 = 45{,}6$, $p < 0{,}001$: los 7 modelos no rinden igual.**

Nemenyi no separa a XGBoost de la logística ni del bosque (diferencia crítica 3,19); Wilcoxon por pares sí: gana los 8 bloques, p de Holm = 0,047.

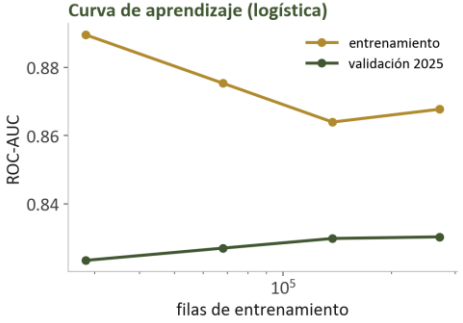
Fuente: tablero, página Modelos; data/modelos/friedman.csv, wilcoxon.csv, rangos.csv

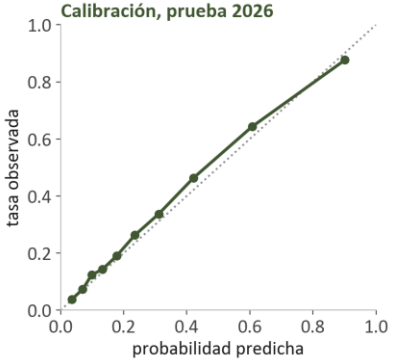
# Sobreajuste bajo: 0,003 de AUC entre interno y externo

## CRITERIO 5 · ANÁLISIS DE ERRORES Y SOBREAJUSTE




El bloque 2025-S1 baja de 0,894 adentro a 0,846 afuera: es el semestre en que cambia la mezcla de 2025 (diapositiva 8), no un modelo que memoriza.




Logística (modelo base): 0,868 en entrenamiento y 0,830 en validación con todas las filas; bien calibrada en 2026 (ECE 0,021).

**Pendiente: evaluación final 2026**

para el mejor modelo en 2026: matriz de confusión por línea de negocio, errores con entidades nuevas frente a conocidas, un falso negativo típico y SHAP (qué variables mueven cada predicción).

Fuente: libro, capítulos 4 y 5: resultados/experimentos/por_bloque.csv, data/modelos/curva_aprendizaje.csv

# El retraso se puede anticipar el día de la liberación

## CRITERIO 6 · CONCLUSIONES

- Se puede ordenar por riesgo al liberar: AUC 0,852 medido hacia adelante en el tiempo, contra 0,670 de la regla del proveedor en 2026.
- La señal está en el plazo, el historial del proveedor y las entidades; ninguna variable sola basta.
- XGBoost gana, pero por una centésima: la logística, más simple y calibrada, es una alternativa defendible.
- El optimizador importa con poco presupuesto; el balanceo casi no mueve el AUC.
- La partición cronológica no es un detalle: al azar, el problema parecería más fácil de lo que es.

| objetivo | cumplido |
|---|---|
| **comparar las 112 con el mismo presupuesto** | sí |
| **costo de las entidades nuevas** | pendiente: 2026 |
| **modelo servido en el tablero** | sí, el modelo base |

**Para el negocio:** marcando el 38 % de las posiciones, el modelo base ve venir el 71 % de las tardías de 2026. El comprador puede concentrar el seguimiento en un tercio de las órdenes.

Fuente: libro, capítulos 2 a 7

# Límites: deriva, etiqueta que madura y una sola empresa

## CRITERIO 6 · LIMITACIONES Y MEJORAS

### Teóricas

Independencia dado x: las posiciones de una orden se parecen; se protege agrupando, no se modela.
Estabilidad: el modelo supone que la relación de 2021-2025 vale en 2026.
Una sola etiqueta binaria desperdicia los días de retraso.

### Empíricas

Deriva: 2025 se distingue de 2021-2024 casi por completo.
La etiqueta de lo más reciente madura tarde (censura al final del periodo).
Maestros en foto actual, no con su vigencia histórica.
Arranque en frío: 11,1 % de proveedores nuevos en 2026.
Una sola empresa: la generalización a otros grupos no está probada.

### Mejoras

Reentrenar por semestre y vigilar la deriva con la validación adversaria.
Efectos aleatorios por proveedor y material para el arranque en frío.
Usar los días de retraso con cortes ordenados (el modelo original del Entregable 3).
Maestros con vigencia por fecha.

Fuente: libro, capítulos 2c, 5 y 7




# Preguntas

Repositorio github.com/ddcarbono/otif-compras-ml

Libro ddcarbono.github.io/otif-compras-ml

Tablero otif-compras-ml.onrender.com

Responden los dos autores (criterio 7: dominio individual).